

Abonnements et concours payants — guide d'intégration mobile

Guide pour l'écran d'abonnement et le verrouillage des concours. Toutes les réponses ci-dessous sont issues d'**appels réels** sur l'environnement de développement.

Référence : issue #244.

1. Ce qui change pour l'application

Le téléchargement d'un concours devient **réservé aux abonnés**. Trois nouveautés côté client :

- un **écran d'abonnement** alimenté par `GET /abonnements/plans` ;
- un **cadenas** sur les concours, piloté par le drapeau `verrouille` ;
- un **403 SUBSCRIPTION_REQUIRED** à traiter en ouvrant l'écran d'abonnement,

pas en affichant une erreur générique.

⚠ Le verrou est éteint au lancement

Le serveur démarre avec `ABONNEMENTS_VERROU_ACTIF=false` : le téléchargement **passe encore**, et le refus qui aurait eu lieu est seulement journalisé. C'est délibéré — livrer le blocage avant qu'il existe un moyen de payer couperait un accès sans proposer d'issue.

L'application doit malgré tout être prête avant la bascule. `verrouille` et `mes-droits` renvoient déjà la vérité (« cet utilisateur n'a pas le droit »), indépendamment du fait que le serveur applique ou non le refus. Développez et testez contre ces valeurs ; le jour de la bascule, rien ne change côté client.

`GET /abonnements/mes-droits` expose l'état du verrou dans `verrou_actif` — utile pour vos propres tests, **pas** pour décider quoi afficher.

Le catalogue reste ouvert

`GET /concours`, `/concours/v1` et `/concours/:id` répondent sans abonnement. Seul `GET /concours/:id/telechargement` est gardé. On ne peut pas demander d'acheter ce qu'on ne voit pas : montrez les concours, avec un cadenas.

Authentification. Tous ces endpoints exigent `Authorization: Bearer <token>`.

Scope pays. `?country=benin` sur les `GET`, `pays` dans le corps des écritures JSON — comme le reste de l'API.

2. Le parcours

```
Liste des concours —
GET /concours → chaque élément porte `verrouille`
verrouille = true → cadenas + CTA « S'abonner »
```

↓ l'utilisateur touche un concours verrouillé

```
Écran d'abonnement —
GET /abonnements/plans → catalogue
POST /abonnements/souscrire → abonnement EN_ATTENTE
```

↓ paiement (hors app pour l'instant)

```
Un administrateur encaisse et active —
l'abonnement passe à ACTIF
```

↓

```
GET /abonnements/mon-abonnement → statut ACTIF
le téléchargement fonctionne
```

3. Le catalogue — GET /abonnements/plans?country=benin

```
[
  {
    "uuid": "574612e9-f59e-433d-8eb8-1da26459df71",
    "code": "MENSUEL",
    "libelle": "Abonnement mensuel",
    "description": "Accès illimité pendant 1 mois",
    "prix": 2000,
    "devise": "XOF",
    "duree_jours": 30,
    "est_actif": true,
    "ordre_affichage": 1
  }
]
```

Trié par `ordre_affichage` puis par prix — **affichez dans l'ordre reçu**, ne retiriez pas côté client, c'est le levier de mise en avant commercial.

Un tableau vide est un état normal, pas une erreur. Les plans sont fermés tant que l'encaissement n'est pas livré. Prévoyez un écran « bientôt disponible » plutôt qu'un spinner infini ou un message d'échec.

`prix` est un **nombre**, pas une chaîne. `duree_jours` sert à afficher la durée sans la déduire du `code`.

4. Souscrire — POST /abonnements/souscrire

```
{ "plan_uuid": "574612e9-f59e-433d-8eb8-1da26459df71", "pays": "benin" }
```

Réponse — 201

```
{
  "uuid": "8960c6bb-4cc3-49da-876e-1b22842e7701",
  "pays": "benin",
  "utilisateur_id": 26746,
  "statut": "EN_ATTENTE",
  "date_debut": null,
  "date_fin": null,
  "montant_paye": 0,
  "devise": "XOF",
  "plan": { "uuid": "574612e9-...", "code": "MENSUEL", "prix": 2000, "duree_jours": 30 }
}
```

EN_ATTENTE n'ouvre aucun droit. L'abonnement ne devient **ACTIF** qu'après encaissement. Tant que l'intégration du paiement n'est pas livrée, c'est un administrateur qui active après réception hors application.

L'écran doit donc afficher un état d'attente explicite — « en attente de confirmation de paiement » — et **surtout pas** annoncer que l'abonnement est actif.

Appeler `souscrire` deux fois de suite ne crée pas deux lignes : la souscription en attente est réutilisée. Inutile de vous prémunir contre le double-tap.

Erreurs

statut	corps	quand
409	<code>{"message":"Ce plan n'est pas disponible"}</code>	plan fermé — rafraîchissez le catalogue
409	<code>{"message":"Vous avez déjà un abonnement actif"}</code>	rien à vendre, renvoyez vers l'écran d'état
404	<code>{"message":"Plan introuvable"}</code>	<code>plan_uuid</code> obsolète

5. L'état courant — GET /abonnements/mon-abonnement?country=benin

Renvoie l'abonnement **actif** ; à défaut le dernier **en attente** ; sinon une réponse **vide**.

```
{
  "uuid": "8960c6bb-...",
  "statut": "ACTIF",
  "date_debut": "2026-09-06T02:05:11.000Z",
  "date_fin": "2026-10-06T02:05:11.000Z",
  "montant_paye": 2000,
  "devise": "XOF",
  "plan": { "code": "MENSUEL", "libelle": "Abonnement mensuel" }
}
```

Les statuts : `EN_ATTENTE` , `ACTIF` , `EXPIRE` , `ANNULE` , `REMBOURSE` .

`statut === "ACTIF"` ne suffit pas. Le passage à `EXPIRE` est écrit par une tâche horaire ; entre l'échéance et son passage, la ligne reste `ACTIF` avec une `date_fin` dépassée. Le serveur, lui, refuse déjà l'accès. Vérifiez les deux : `` dart final actif = a?.statut == 'ACTIF' && a!.dateFin.isAfter(DateTime.now()); `` Se fier au seul statut afficherait « abonné » à quelqu'un que le serveur vient de bloquer — le pire des messages.

`GET /abonnements/mes-abonnements` donne l'historique paginé.

6. Les droits — `GET /abonnements/mes-droits?country=benin`

```
{
  "verrou_actif": false,
  "droits": {
    "CONCOURS_DOWNLOAD": { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" },
    "EPREUVE_VIEW": { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" },
    "EXAMEN_NAT_VIEW": { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" },
    "KETSIA_AI": { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" },
    "AI_STATS": { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" }
  }
}
```

Un appel au lancement de session suffit ; rafraîchissez après une activation. `reason` peut valoir `SUBSCRIBED` , `ADMIN` , `SUBSCRIPTION_REQUIRED` — et `FREE_QUOTA / QUOTA_EXCEEDED` quand les quotas gratuits arriveront (#245), avec alors un objet `quota: { used, limit }` .

Seules `CONCOURS_DOWNLOAD` est appliquée aujourd'hui. Les quatre autres sont publiées pour que vous puissiez préparer les écrans ; ne bloquez rien sur leur base pour l'instant.

7. Les concours verrouillés

Chaque concours porte désormais `verrouille` :

```
{ "id": 98, "titre": "INSTI Bénin – Concours Ingénieur", "annee": 2024, "verrouille": true }
```

Présent sur `GET /concours` , sur `GET /concours/:id` et sur chaque instance des groupes de `GET /concours/v1` .

`verrouille: true` → cadenas sur la vignette, et le bouton de téléchargement ouvre l'écran d'abonnement au lieu de lancer la requête.

Le refus — `GET /concours/:id/telechargement`

```
{
  "statusCode": 403,
  "error": "SUBSCRIPTION_REQUIRED",
  "message": "Un abonnement actif est requis pour accéder à cette ressource.",
  "feature": "CONCOURS_DOWNLOAD"
}
```

`error` est le **contrat machine** : branchez-vous dessus pour ouvrir l'écran d'abonnement. `message` est destiné à l'affichage. `feature` dit ce qui a été refusé — utile quand #245 étendra le verrou aux épreuves.

Traitez le 403 même si vous respectez `verrouille` : un abonnement peut expirer entre l'affichage de la liste et le tap.

8. Exemple Flutter

```
class AbonnementsApi {
  AbonnementsApi(this._client, {required this.baseUrl, required this.token, required this.pays});

  final http.Client _client;
  final String baseUrl, token, pays;

  Map<String, String> get _entetes => {
    'Authorization': 'Bearer $token',
    'Content-Type': 'application/json',
  };

  Future<List<Plan>> plans() async {
    final r = await _client.get(Uri.parse('$baseUrl/abonnements/plans?country=$pays'), headers: _entetes);
    final liste = jsonDecode(r.body) as List;
    // Une liste vide est un état normal : les plans ne sont pas encore ouverts.
    return liste.map((e) => Plan.fromJson(e)).toList();
  }

  Future<Abonnement?> monAbonnement() async {
    final r = await _client.get(Uri.parse('$baseUrl/abonnements/mon-abonnement?country=$pays'), headers: _entetes);
    if (r.body.trim().isEmpty || r.body == 'null') return null;
    return Abonnement.fromJson(jsonDecode(r.body));
  }

  Future<Abonnement> souscrire(String planUuid) async {
    final r = await _client.post(
      Uri.parse('$baseUrl/abonnements/souscrire'),
      headers: _entetes,
      body: jsonEncode({'plan_uuid': planUuid, 'pays': pays}),
    );
    if (r.statusCode ~/ 100 != 2) throw ApiException(messageErreur(jsonDecode(r.body)));
    return Abonnement.fromJson(jsonDecode(r.body));
  }
}

class Abonnement {
  final String uuid, statut;
  final DateTime? dateDebut, dateFin;

  /// `statut == 'ACTIF'` NE SUFFIT PAS : la bascule vers EXPIRE est écrite par
  /// une tâche horaire, donc une ligne peut rester ACTIF avec une date_fin
  /// dépassée alors que le serveur refuse déjà l'accès.
  bool get estActif =>
    statut == 'ACTIF' && dateFin != null && dateFin!.isAfter(DateTime.now());

  bool get enAttente => statut == 'EN_ATTENTE';
}
```

Traiter le refus

```
Future<void> telechargerConcours(Concours c) async {
  if (c.verrouille) return ouvrirEcranAbonnement(); // évite un aller-retour inutile

  final r = await client.get(
    Uri.parse('$baseUrl/concours/${c.id}/telechargement?country=$pays'),
    headers: _entetes,
  );

  if (r.statusCode == 403) {
    final corps = jsonDecode(r.body);
    // Contrat machine, pas le texte : le message peut être reformulé côté serveur.
    if (corps['error'] == 'SUBSCRIPTION_REQUIRED') return ouvrirEcranAbonnement();
    if (corps['error'] == 'QUOTA_EXCEEDED') return ouvrirEcranAbonnement(quota: corps['quota']);
  }
  // ... enregistrer le fichier
}
```

9. Rappels

Le verrou est éteint aujourd'hui, l'app doit être prête quand même. `verrouille` et `mes-droits` disent déjà la vérité ; développez contre eux.

Une liste de plans vide n'est pas une erreur. C'est l'état attendu tant que l'encaissement n'est pas livré.

`EN_ATTENTE` n'ouvre aucun droit. N'annoncez jamais un abonnement actif sur la seule foi de la réponse de `souscrire` .

Vérifiez `date_fin` , pas seulement `statut` . Un `ACTIF` échu existe entre l'échéance et le passage horaire de la tâche d'expiration.

Branchez-vous sur `error` , pas sur `message` . Le texte est fait pour être lu par l'utilisateur et peut changer ; `SUBSCRIPTION_REQUIRED` est le contrat.

Le paiement n'est pas encore intégré. L'activation est faite par un administrateur après encaissement hors application. Le parcours de paiement in-app arrivera avec l'intégration du prestataire — l'écran d'abonnement doit pouvoir l'accueillir sans être reconstruit.